2. El modelo

Todo lo de `src/model/`. Aquí no hay ventanas: solo números, colores y una imagen.

Orden recomendado: `Fire` (los números) → `ColorTargetDTO` y `Palette` (los colores) → `Render` (une los dos) → `Model` (lo pone en marcha).

Fire: las temperaturas

`Fire.java`

Los datos

```
private final int [][]temperatures;   // línea 13
```

Es una tabla de `altura` filas × `ancho` columnas (200 × 320). Cada celda es la temperatura de un punto, de 0 a 255.

- **Fila 0** = arriba de la pantalla.
- **Última fila** (`altura - 1`) = la **base**, donde nace el fuego.

Al crearla (`Fire.java:19`) todo vale 0, o sea, apagado.

Paso 1: `sparksGeneration()` — chispas

`Fire.java:26`

Recorre la base de izquierda a derecha. Cada punto tiene un **40%** de probabilidad de encenderse (se pone a 255).

```
if (random.nextInt(100) < sparksPercentage)   // nextInt(100) da 0..99
    temperatures[base][x] = 255;
```

`nextInt(100)` da un número de 0 a 99. Si sale menor que 40 (o sea 0..39, un 40% de los casos), se enciende.

Paso 2: `coolPointsGeneration()` — puntos fríos

`Fire.java:36`

Lo mismo, pero **apaga** (pone a 0) un 30% de la base. Sin esto la base sería una línea blanca fija. Con esto parpadea, y por eso las llamas salen irregulares.

Paso 3: `updateFlames()` — el calor sube

`Fire.java:47`

Es el corazón del efecto. Recorre las filas **de abajo hacia arriba** (empieza en `altura - 2`, la fila justo encima de la base). Cada punto pasa a ser la **media** de cuatro valores:

```
                [y][x]            ← el propio punto (actual)

        [y+1][x-1] [y+1][x] [y+1][x+1]   ← los 3 de la fila de abajo
          (left)   (center)   (right)
```

```
int t = (actual + left + right + center)/4 - random.nextInt(2+1);   // línea 58
temperatures[y][x] = Math.max(t, 0);
```

Ejemplo con números. Un punto con `actual = 0`, y debajo `left = 255`, `center = 255`, `right = 0`:

1. Media: (0 + 255 + 0 + 255) / 4 = **127**
2. `random.nextInt(3)` da 0, 1 o 2. Si sale 2: 127 − 2 = **125**
3. El punto pasa de 0 a 125.

Por eso el calor sube: lo de abajo "contagia" a lo de arriba. Y por eso se apaga a medida que sube: se divide entre 4 y además se resta algo al azar, así que el calor se va perdiendo. Las llamas acaban en punta.

Los bordes. Para el punto de la primera columna, `x - 1` sería `-1` y se saldría del array. Por eso se usa:

```
Math.max(x-1, 0)         // nunca baja de 0
Math.min(x+1, ancho-1)   // nunca pasa de la última columna
```

Que no sea negativo. `Math.max(t, 0)` evita temperaturas negativas, porque la paleta solo tiene colores de 0 a 255.

> Nota: el atributo `randomizer` (`Fire.java:8`) no se usa en ningún sitio.

ColorTargetDTO: un color clave

`ColorTargetDTO.java`

Es un DTO: guarda datos y nada más. Representa una frase: **"a esta temperatura, el color es este"**.

```
new ColorTargetDTO(20, 224, 58, 0, 255)
//                 |    |    |  |   └─ alpha (transparencia, 255 = opaco)
//                 |    |    |  └──── azul
//                 |    |    └─────── verde
//                 |    └──────────── rojo
//                 └───────────────── temperatura
```

Ojo al orden: temperatura, **rojo, verde, azul**, alpha.

Todo es `final` y solo tiene getters: una vez creado, no cambia.

Palette: de temperatura a color

`Palette.java`

No escribimos los 256 colores a mano. Damos **4 colores clave** y la paleta rellena lo que hay en medio mezclando.

Los colores clave

`Palette.createPalette()`

Temperatura	R	G	B	A	Color
0	0	0	0	0	negro transparente
20	224	58	0	255	rojo anaranjado
80	255	208	32	255	amarillo
255	255	255	255	255	blanco

La tabla

```
private final int[] colors = new int[256];   // línea 9
```

`colors[t]` guarda el color de la temperatura `t`. Se rellena **una sola vez**, al crear la paleta. Después, consultar un color (`color()`) es solo leer la tabla, y por eso es rápido.

El constructor, paso a paso

`Palette.java:11`

Hay 4 colores clave, así que hay **3 tramos**: 0→20, 20→80 y 80→255. Cada tramo es una vuelta del bucle exterior:

```
for (int i = 0; i < targets.size()-1; i++) {
    ColorTargetDTO from = targets.get(i);      // inicio del tramo
    ColorTargetDTO to   = targets.get(i+1);    // final del tramo
```

El `-1` es necesario porque se usa `i+1`. Con 4 targets, `i` llega como mucho a 2, y `i+1` es 3, que existe. (Sin el `-1`, `i+1` llegaría a 4 y daría `IndexOutOfBoundsException`.)

Para cada tramo, el bucle interior recorre todas las temperaturas que hay entre `from` y `to`.

El `ratio`

`Palette.java:21`

```
double ratio = (double)(t - from.getTemperature()) / range;
```

Dice **en qué punto del tramo estás**, de 0 a 1:

- `0` → estás justo en `from`
- `1` → estás justo en `to`
- `0.5` → estás a la mitad

Con el tramo 0→20 (`range = 20`):

t	t − from	ratio
0	0	0/20 = **0**
5	5	5/20 = **0.25**
10	10	10/20 = **0.5**
20	20	20/20 = **1**

El `(double)` es importante: sin él, `5/20` en enteros daría `0`, no `0.25`.

`mix`: el valor entre dos números

`Palette.java:41`

```
return (int) Math.round(v1 + (v2-v1)*ratio);
```

Te da un número entre `v1` y `v2`, tan cerca de uno o del otro como diga el `ratio`:

1. `v2 - v1` = la distancia entre los dos.

2. `* ratio` = la parte de esa distancia que recorres.
3. `v1 +` = partes del inicio y sumas ese recorrido.

`Math.round` redondea y `(int)` lo convierte en entero, porque los colores no llevan decimales.

Ejemplo completo: temperatura 50

Está en el tramo 20→80 (`from` = rojo anaranjado, `to` = amarillo). `range = 60`.

`ratio = (50 − 20) / 60 = 0.5`

Canal	from	to	Cálculo	Resultado
rojo	224	255	224 + (255−224) × 0.5 = 239.5	**240**
verde	58	208	58 + (208−58) × 0.5	**133**
azul	0	32	0 + (32−0) × 0.5	**16**
alpha	255	255	255 + 0 × 0.5	**255**

El color de la temperatura 50 es (240, 133, 16, 255): un naranja entre el rojo y el amarillo.

Empaquetar el color

`Palette.java:32`

```
colors[t] = new Color(red, green, blue, alpha).getRGB();
```

`getRGB()` junta los 4 canales en **un solo** `int` (formato ARGB). Es el formato que necesita luego la imagen.

Render: de números a imagen

`Render.java`

Une `Fire` (los números) con `Palette` (los colores) y produce una `BufferedImage`, que es lo que se puede dibujar.

```
this.image = new BufferedImage(fire.getAncho(), fire.getAltura(), BufferedImage.TYPE_INT_ARGB);
```

La imagen tiene **un píxel por cada punto de temperatura** (320 × 200). Se crea una sola vez y se reutiliza.

`getImage()`

`Render.java:18`

```
for (int y = 0; y < fire.getAltura(); y++) {
    for (int x = 0; x < fire.getAncho(); x++) {
        image.setRGB(x, y, palette.color(temperatures[y][x]));
    }
}
```

Para cada punto: lee su temperatura → la paleta le da un color → se pinta ese píxel.

Ojo con el orden: la tabla se lee como `[y][x]` (fila, columna), pero `setRGB` recibe `x, y` (columna, fila). Es al revés, y es un fallo muy típico.

Model: lo pone todo en marcha

`Model.java`

Crea `Fire` y `Render` (`Model.java:14`) y hace avanzar el fuego en su propio hilo.

El hilo

`Model.start()`

```
Thread thread = new Thread(this, "Model-Fire");
thread.setDaemon(true);
thread.start();
```

- `new Thread(this, ...)`: el `this` es importante. Le dice al hilo **qué ejecutar**: el método `run()` de esta clase. Sin el `this`, el hilo estaría vacío y no haría nada.
- `setDaemon(true)`: el hilo se cierra solo cuando se cierra el programa.
- `start()`: lo arranca.

`run()`: el bucle

`Model.run()`

```
while (true) {
    update();
    Thread.sleep(frameMS);   // 33 ms
}
```

Repite para siempre: avanzar un paso y esperar 33 ms. 1000 / 33 ≈ **30 pasos por segundo**.

`update()`: un paso de animación

`Model.update()`

```
fire.sparksGeneration();      // 1) enciende la base
fire.coolPointsGeneration();  // 2) apaga parte de la base
fire.updateFlames();          // 3) sube el calor
```

El orden importa: primero se prepara la base, y luego el calor sube desde ella.

`getImage()`

`Model.getImage()` simplemente le pide la imagen a `Render`.